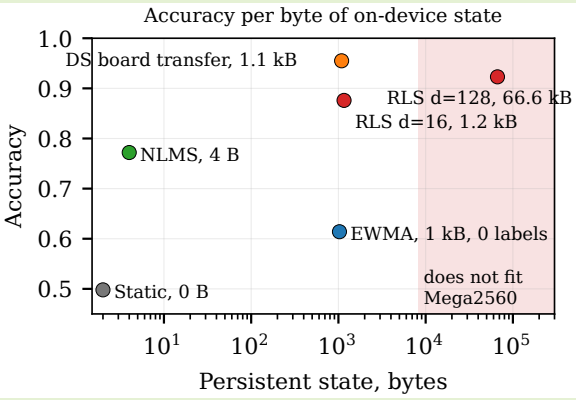
Drift Compensation and Calibration Transfer for Electronic Noses per Byte and per Microsecond on Microcontrollers

Oleksandr Kuznetsov, *Member, IEEE*

***Abstract*— Like its biological prototype, an electronic nose relies on an array of cross-sensitive receptors whose responses drift over time and differ between arrays. Compensation methods are usually compared by accuracy alone, although a low-cost instrument must run them on a microcontroller with a few kilobytes of memory. We measure how much accuracy linear drift-compensation and calibration-transfer methods deliver per byte of persistent state and per microsecond. On two public metal-oxide array datasets (36-month drift; five twin boards), under a protocol frozen before the runs, we compare unlabelled running standardisation, normalized least mean squares, recursive least squares (RLS), block least squares and direct standardisation (DS). All kernels and a streaming feature extractor were executed in float32 on Mega2560, ESP32-C3 and ESP32-S3 boards and reproduced the host predictions on every fixture sample. Without labels, standardisation raises drift accuracy from 0.498 to 0.614 with 1 028 bytes. With 20 references per batch, RLS reaches 0.923 but needs 66.6 kB, whereas compact-feature RLS and block least squares reach 0.876 and 0.898 with 1.2–1.3 kB. Between twin boards, compact DS reaches 0.955 with 1 088 bytes and 16 transfer pairs; its fit takes 0.27 s on Mega2560. Even on an 8-bit processor, memory rather than time decides which method can be deployed.**



***Index Terms*— Calibration transfer, drift compensation, electronic nose, embedded systems, gas sensor arrays, microcontrollers, recursive least squares.**

I. Introduction

THE electronic nose was proposed as a model of the mammalian olfactory system: instead of one selective receptor per odorant, an array of broadly tuned sensors produces a response pattern that is recognised by a trained classifier [1]. Metal-oxide (MOX) gas sensors are the most common receptors in such arrays because they are cheap, sensitive and easy to read out. Their weakness is equally well known. The response of a MOX sensor changes over months because of ageing, poisoning and environmental history, and two nominally identical arrays respond differently to the same gas [2]. A classifier trained once, therefore, gradually loses accuracy on the same instrument, and a classifier trained on one instrument does not transfer to another without correction.

Both problems have been studied extensively. For long-term drift, component correction [3], orthogonal signal correction [4], common principal components [5], classifier ensembles [6] and domain-adaptation extreme learning machines [7] have been proposed, and a public 36-month dataset [6], [8] has become the common benchmark. For instrument-to-instrument variation, direct standardisation (DS), originally developed for spectrometers [9], has been applied to twin MOX arrays with only a few transfer samples [10]. The comparisons in this literature are, however, made on a workstation, and the criterion is classification accuracy. The cost of running a method on the instrument itself is not reported. A recent review of drift compensation for MOX arrays names the trade-off between performance and efficiency, and models that can actually be deployed, among the open problems [11]. The closest embedded work we found trains a compact neural network for drift compensation offline and quantizes it for inference [12]; on-device adaptation, persistent memory and execution time on a microcontroller are not measured there.

For a low-cost electronic nose, this omission matters. An Arduino-class 8-bit microcontroller has 8 kB of SRAM, and even ESP32-class devices share a few hundred kilobytes with communication stacks. A compensation method that updates its model from reference measurements must keep its state between samples, and this state may be much larger than the classifier itself. Studies of on-device learning make the

O. Kuznetsov is with the Department of Theoretical and Applied Sciences, eCampus University, Italy, and with the Department of Intelligent Software Systems and Technologies, School of Computer Science and Artificial Intelligence, V.N. Karazin Kharkiv National University, Ukraine (e-mail: oleksandr.kuznetsov@uniecampus.it; ORCID: 0000-0003-2331-6326).

same observation for neural networks: update memory must be counted separately from inference memory [13]. At the same time, the timing requirement of a gas sensor array is mild, because one measurement cycle lasts minutes. It is therefore not obvious in advance whether memory or time limits the choice of method on a given processor.

The aim of this work is to quantify, on public recordings of real MOX arrays and on physical microcontrollers, how much accuracy simple drift-compensation and calibration-transfer methods deliver per byte of persistent state and per microsecond of execution. We deliberately restrict ourselves to methods that update a linear classifier or a linear feature map, because they can be written as short float32 kernels with exactly known state, and we do not compete with ensembles or deep models on accuracy. The contribution of the paper is as follows.

1) We compare eight compensation methods and two combinations on the 36-month drift dataset, and eight methods including three calibration-transfer methods on five twin boards, under a protocol whose data splits, hyperparameter grids and seeds were fixed before the experiments, with exact byte counts of the persistent state.
2) We implement the methods that matter for deployment, together with a streaming feature extractor, as float32 C kernels and verify on three boards (AVR, RISC-V and Xtensa with a floating-point unit) that they reproduce the host predictions on every fixture sample.
3) We report measured time per call on the three boards and combine it with the accuracy and memory results into a deployment map, which shows that memory rather than time decides what can run on an 8-bit device.

II. Data and Common Model

A. Datasets

We use two public datasets of the UCI repository. The first (D1) is the gas sensor array drift dataset [6], [8], [14], in the version with concentrations [15]. It contains 13 910 recordings of six gases (ammonia, acetaldehyde, acetone, ethylene, ethanol and toluene) measured by an array of 16 MOX sensors over 36 months and grouped into ten batches. Each recording is described by 128 features, eight per sensor: the steady-state resistance change ΔR, its normalised form, and the maxima and minima of an exponential moving average of the resistance derivative for three smoothing constants, as introduced in [16]. Batches are unbalanced (161 to 3 613 recordings), and toluene is absent from batches 3–5. The second dataset (D2) contains recordings of five twin arrays of eight Figaro MOX sensors exposed to carbon monoxide, ethanol, ethylene and methane at ten concentration levels [10], [17]. Its 640 raw recordings last 600 s at 100 Hz; boards 1–3 were measured on four days and boards 4 and 5 on two. We assume that repetition indices correspond to the order of measurement days. All archives were verified by SHA-256 digests before use.

B. Features and classifier

For D1 we use either all 128 features ($d = 128$) or the 16 ΔR features ($d = 16$). For D2, the features are computed by our own streaming extractor (Section III-A), and we use either all eight features of each sensor ($d = 64$) or only ΔR and $\Delta R/R_0$ ($d = 16$). Each feature is compressed by the signed logarithm $s(x) = \operatorname{sign}(x)\log(1 + |x|)$ and standardised with the mean and standard deviation of the source data, giving the vector $\boldsymbol{z} \in \mathbb{R}^d$.

All methods share one classifier. With $\boldsymbol{a} = [\boldsymbol{z}^T, 1]^T$, the predicted class is

$$\hat{c} = \arg\max_c \ \boldsymbol{a}^T \boldsymbol{w}_c, \qquad \boldsymbol{W} = [\boldsymbol{w}_1, \ldots, \boldsymbol{w}_C] \in \mathbb{R}^{(d+1)\times C}, \tag{1}$$

where $\boldsymbol{W}$ is obtained by ridge regression of the source samples onto one-hot class targets. Such a least-squares classifier is weaker than a support-vector machine, but it has an important advantage for this study: the sequential and block estimators described below update exactly the model that is deployed. In float32, the head occupies $4C(d + 1)$ bytes, i.e., 408 B for $d = 16$ and 3 096 B for $d = 128$ in D1. For concentration regression (Section IV), one output per gas predicts $\log_{10}$ of the concentration in ppmv.

III. Compensation Methods and State Accounting

We count as *persistent state* the bytes that a method must keep between two samples in addition to the frozen head, assuming float32 values and one byte per stored class label. Scratch memory that is released after a call is excluded and is reported for the firmware separately. Methods that modify the head (NLMS, RLS and block least squares) must also keep the head itself in RAM, whereas the other methods can leave it in flash memory; we return to this point in Section VII.

M0, static head. The source model is applied without change; the state is 0 B.

M1, EWMA standardisation. Without any labels, the running mean $\boldsymbol{m}$ and variance $\boldsymbol{v}$ of the incoming features replace the source statistics:

$$\boldsymbol{m}_t = \boldsymbol{m}_{t-1} + \alpha(\boldsymbol{z}_t - \boldsymbol{m}_{t-1}), \tag{2}$$

$$\boldsymbol{v}_t = (1 - \alpha)\big(\boldsymbol{v}_{t-1} + \alpha(\boldsymbol{z}_t - \boldsymbol{m}_{t-1})^{\odot 2}\big), \tag{3}$$

and the head receives $(\boldsymbol{z}_t - \boldsymbol{m}_t) \oslash (\boldsymbol{v}_t + \epsilon)^{\odot 1/2}$. The method resembles the adaptation of a receptor to a constant background, and its state is $8d + 4$ bytes.

M2–M4, other unsupervised and paired corrections. Adaptive class centroids (M2, $4Cd + 4$ B), correction along r drift components estimated from references in the spirit of [3] (M3, $4rd + 4$ B), and a per-feature gain and offset fitted to references (M4, $8d$ B) complete the set of feature-level corrections.

M5, NLMS on the head. For each reference $(\boldsymbol{z}, c)$ with one-hot target $\boldsymbol{t}$, the head is updated as [18]

$$\boldsymbol{W} \leftarrow \boldsymbol{W} + \frac{\mu}{\|\boldsymbol{a}\|^2 + \epsilon}\, \boldsymbol{a}\,(\boldsymbol{t} - \boldsymbol{W}^T\boldsymbol{a})^T. \tag{4}$$

Since the head is updated in place, only the step size is stored, i.e., 4 B.

M6, RLS on the head. Exponentially weighted RLS keeps the $(d + 1) \times (d + 1)$ inverse correlation matrix $\boldsymbol{P}$, initialised

from the source fit as $\boldsymbol{P}_0 = (\boldsymbol{A}^T\boldsymbol{A} + \lambda\boldsymbol{I})^{-1}$ so that the first update continues the ridge solution instead of restarting it. With forgetting factor β,

$$\boldsymbol{g} = \frac{\boldsymbol{P}\boldsymbol{a}}{\beta + \boldsymbol{a}^T\boldsymbol{P}\boldsymbol{a}}, \qquad \boldsymbol{W} \leftarrow \boldsymbol{W} + \boldsymbol{g}(\boldsymbol{t} - \boldsymbol{W}^T\boldsymbol{a})^T, \quad (5)$$

$$\boldsymbol{P} \leftarrow \beta^{-1}(\boldsymbol{P} - \boldsymbol{g}\boldsymbol{a}^T\boldsymbol{P}). \quad (6)$$

The state is $4(d+1)^2+4$ B: 1 160 B for $d = 16$, but 66 568 B for $d = 128$.

M7, block least squares. The k references of one calibration session are buffered, and the head is refitted once by ridge regression shrunk towards the current head,

$$\boldsymbol{W} \leftarrow \arg\min_{\boldsymbol{X}} \ \|\boldsymbol{T} - \boldsymbol{A}_k\boldsymbol{X}\|_F^2 + \gamma\|\boldsymbol{X} - \boldsymbol{W}\|_F^2, \quad (7)$$

which is solved by a Cholesky factorisation of a $(d+1)\times(d+1)$ matrix. The buffer takes $k(4d+1)$ B.

M8 and M9, direct standardisation. For calibration transfer, k pairs of feature vectors of the same gas and concentration, measured on the master and on the slave board, give the affine map $\boldsymbol{F}$ from slave to master features. We fit it by (7) with the master features as targets and the identity as prior, and store $(d+1)d$ float32 values (1 088 B for $d = 16$). The per-channel variant M9 maps each sensor separately with a band of half-width w and stores $4d(2w+1)$ B.

Two combinations, M1+M5 and M1+M7, apply the labelled update after the unsupervised standardisation.

A. Streaming feature extractor

The D1 features were computed by the dataset authors from complete recordings. For D2 we compute features of the same kind from the raw resistance traces with an extractor that processes one sample at a time and keeps a constant state, as it would on an instrument. During the first 30 s, before the gas reaches the chamber, it accumulates the baseline R_0 of each sensor; afterwards it tracks the minimum resistance $R_{\min}$. For each of three smoothing constants $\alpha \in \{0.001, 0.01, 0.1\}$, an exponential moving average of the first difference of the resistance is updated, and its maximum and minimum are kept. At the end of a recording, the eight features of each sensor are $\Delta R = R_0 - R_{\min}$, $\Delta R/R_0$ and the six extrema. For eight sensors, the state takes 426 B on the Mega2560 and 428 B on the ESP32 boards, which differ only in structure padding. In float32, the C extractor and the host extractor give bit-identical features.

IV. Protocol

The protocol, including data splits, hyperparameter grids, selection rules and random seeds, was written down and committed to the public repository before each experiment was run (E1/E2 and E3/E4 separately); results were not used to change it. Hyperparameters were selected only on D1 batch 2 and on the D2 board pair $1 \to 2$, which are excluded from the reported averages.

E1, static settings. The head is trained on batch 1 and tested on batches 2–10 (setting 1), or trained on batch $b-1$ and tested on batch b (setting 2); a multinomial logistic head is included for reference.

TABLE I
Boards, Toolchains and Firmware Size

Board	Core, clock	FPU	RAM used (B)	Flash used (B)
Mega2560	AVR, 16 MHz	no	5 497 / 8 192	67 378
ESP32-C3	RV32IMC, 160 MHz	no	84 812 / 327 680	348 758
ESP32-S3	Xtensa LX7, 240 MHz	single	89 412 / 327 680	362 101

Arduino framework via PlatformIO; avr-gcc 7.3.0 for Mega2560, GCC 8.4.0 (esp-2021r2-patch5) for the ESP32 boards, default `-Os`. RAM includes a static arena of 4 600 B on Mega2560 and 70 724 B on the ESP32 boards.

E2, streaming drift compensation (main experiment). Batches 2–10 of D1 are processed in order, and the state of every method carries over from batch to batch. In each batch, a seeded permutation fixes the sample order; the first $k \in \{0, 1, 5, 20\}$ samples are labelled references, i.e., one calibration session, and the remaining samples are classified in order without labels. Accuracy is averaged over batches 3–10 and 20 permutation seeds. The head ridge parameter and the method hyperparameters were selected from fixed grids by the mean accuracy on batch 2 with five separate tuning seeds and $k = 5$.

E3, concentration under drift. With the gas identity known, one regression head per gas predicts $\log_{10}$ ppmv; the stream, budgets and selection follow E2, and the metric is the root-mean-square error (RMSE) averaged over gases.

E4, calibration transfer and day-to-day drift. The head is trained on all recordings of a master board and applied to a slave board, for all 20 ordered pairs. For $k \in \{0, 2, 4, 8, 16, 40\}$, k of the 40 gas–level conditions are drawn per seed, and their first-day recordings on both boards form transfer pairs (M4, M8, M9) or labelled references (M5–M7). All other recordings of the slave board are evaluated. We report the mean and the worst of the 19 pairs other than the tuning pair, over 20 seeds. Within boards 1–3, we also train on the first day and test on the following ones.

E5, microcontrollers. The kernels of Section III and the feature extractor were written in C and compiled with `-fno-fast-math -ffp-contract=off` both for the host and for the boards. A fixture of real samples is stored in flash: 64 and 32 recordings of D1 batch 7 for $d = 16$ and $d = 128$ with the frozen E2 heads and hyperparameters, 20 references and 44 test recordings for the block refit, 16 transfer pairs and 64 test recordings from D2 boards $1 \to 2$ for DS, and 512 samples of one raw D2 recording for the extractor. Each kernel first runs once and is checked against the host reference output (identical predictions on every sample, checksums within 10^{-4} relative, features within 10^{-5}); only a kernel that passes is timed ten times with `micros()`. Each board was flashed once and measured in two sessions. Table I lists the boards and builds.

Statistics. Accuracy is reported with 95% bootstrap intervals over seeds in the repository; in the paper we test selected differences by two-sided Wilcoxon signed-rank tests over the eight test batches (E2) or the 19 board pairs (E4), with the Holm correction over the family of nine tests [19]. With eight batches, the smallest attainable two-sided p is 0.0078.

V. Results on the Host

A. Drift over 36 months

In the static settings of E1, a head trained on batch 1 reaches only 0.542 on batches 2–10 with all 128 features and 0.468 with 16 features, whereas retraining on the previous batch gives 0.783 and 0.606. Standardising each test batch with its own statistics raises setting 1 to 0.684 and 0.599. These figures confirm the strong drift of D1 and serve as a frame for the streaming results.

Table II and Fig. 1(a) summarise E2. Without any labels, EWMA standardisation improves the static head from 0.434 to 0.541 at $d = 16$ (132 B) and from 0.498 to 0.614 at $d = 128$ (1 028 B). The improvement is seen in six and seven of the eight test batches, respectively, but with eight batches it is not significant after correction ($p_{\mathrm{Holm}} = 0.23$ and 0.22). Adaptive centroids (M2), component correction (M3) and per-feature affine correction (M4) give small or negative changes in this protocol and are omitted from the table.

Labels change the picture more strongly. With 20 references per batch, NLMS at $d = 128$ reaches 0.772 with only 4 B of additional state and is better than the static head in all eight batches ($p_{\mathrm{Holm}} = 0.047$). RLS reaches the highest accuracy, 0.923, and exceeds NLMS in every batch ($p_{\mathrm{Holm}} = 0.047$), but its 66.6 kB covariance matrix exceeds the SRAM of the Mega2560 eightfold. On the 16 compact features, the same RLS needs 1 160 B and reaches 0.876, and block least squares reaches 0.898 with a 1 300 B buffer; the difference between these two is not significant ($p_{\mathrm{Holm}} = 0.98$). One reference per batch is not sufficient: block least squares even collapses to 0.243 at $d = 16$, because its shrinkage $\gamma = 0.1$ was selected at $k = 5$.

B. Concentration under drift

For regression (Table III), the static heads have an RMSE of 0.31–0.34 in $\log_{10}$ ppmv, i.e., a typical factor of about two in concentration. Without labels, EWMA standardisation helps at $d = 128$ (0.241) but hurts at $d = 16$ (0.352): within one gas, the incoming concentration mix differs from that of the source, and standardisation removes part of the concentration signal itself. With five references, NLMS at $d = 128$ reduces the RMSE to 0.154 with 4 B of state, and block least squares to 0.138. RLS with the selected forgetting factor $\beta = 0.8$, which lies on the edge of the grid, becomes unstable for some gases at $k = 20$ (toluene and ammonia), which is the known covariance wind-up of RLS with strong forgetting and few informative samples.

C. Calibration transfer between twin boards

Board-to-board variation is the more severe problem in D2 (Table IV, Fig. 1(b)). A master head applied to a slave board reaches 0.831 on average, with a worst pair of 0.65, whereas within a board the accuracy stays between 0.87 and 0.95 over up to four measurement days. EWMA standardisation of the slave stream, again without labels, raises transfer accuracy to 0.895 with 132 B and is better than the static head in 15 of 19 pairs ($p_{\mathrm{Holm}} = 0.002$). DS on the compact features reaches 0.925 with eight transfer pairs and 0.955 with sixteen, with a worst pair of 0.92; it exceeds EWMA in 15 pairs ($p_{\mathrm{Holm}} = 0.016$) and RLS on the head in 13 pairs ($p_{\mathrm{Holm}} = 0.020$). DS on all 64 features needs 16.6 kB and is not better (0.946, $p_{\mathrm{Holm}} = 0.98$ for the difference). The paired affine and per-channel corrections (128–192 B) stay at 0.89–0.90, at the level of EWMA, which needs no transfer samples at all.

VI. Execution on Microcontrollers

A. Agreement with the host

All six sessions (two per board) passed validation. On every board, each kernel reproduced the host predictions on every fixture sample, and the feature extractor reproduced the host features exactly. The checksums of the updated heads and of the DS map differed from the host by at most 3×10^{-6} relative, which reflects differences in the float32 rounding of the target libraries; the predictions were not affected. The source files on the boards were identical to the repository version. RLS at $d = 128$ does not fit the Mega2560 and is reported there as skipped.

B. Time per call

Table V and Fig. 2 give the median time per call. The two sessions on each board differ by at most 0.13%, and so do the ten repetitions within a session; the largest spread belongs to the single block refit on the ESP32-S3 (796 versus 797 μs), where the resolution of the timer becomes visible.

Three observations follow from these measurements. First, all kernels are fast in relation to the measurement cycle of a MOX array, which lasts minutes. The slowest per-sample kernel on the Mega2560, classification with an NLMS update at $d = 128$, takes 75 ms, and a DS fit from 16 pairs takes 0.27 s and is executed once per calibration session. The streaming extractor, which must keep up with the 100 Hz sampling, occupies 12.7% of the Mega2560, 1.0% of the ESP32-C3 and 0.05% of the ESP32-S3.

Second, label-driven adaptation is cheap compared with inference itself. An NLMS update increases the time of a static classification by a factor of 1.5–1.8 on all boards at 4 B of state, and RLS at $d = 16$ by a factor of 2.9–4.1. The preprocessing with one logarithm per feature dominates the static classification on processors without a floating-point unit.

Third, the ESP32-C3, which has no floating-point unit, is 10.7–14.1 times faster than the Mega2560, which is close to the ratio of their clock frequencies. The ESP32-S3 with a single-precision unit is a further 15–27 times faster. Since the instruction set, memory system and compiler also differ between the boards, these ratios describe the tested builds and do not isolate the effect of the floating-point unit.

VII. Discussion

Taken together, the results give a simple deployment map. On an 8-bit microcontroller with 8 kB of SRAM, every method on compact features fits, and the choice depends on what references are available. Without any labels, EWMA standardisation is the obvious first step, as it costs 132 B

TABLE II

DRIFT COMPENSATION ON THE GAS SENSOR ARRAY DRIFT DATA: ACCURACY (BATCHES 3–10, 20 SEEDS) AND PERSISTENT STATE

	$d = 16$					$d = 128$				
Method	$k = 0$	1	5	20	State (B)	$k = 0$	1	5	20	State (B)
Static head	0.434	0.434	0.434	0.433	0	0.498	0.498	0.498	0.498	0
EWMA standardisation	0.541	0.541	0.541	0.541	132	0.614	0.614	0.614	0.614	1 028
NLMS on head	–	0.447	0.479	0.574	4	–	0.579	0.676	0.772	4
RLS on head	–	0.447	0.631	0.876	1 160	–	0.634	0.792	0.923	66 568
Block OLS on head	–	0.243	0.614	0.898	1 300	–	0.481	0.721	0.855	10 260
EWMA + NLMS	–	0.546	0.557	0.575	136	–	0.645	0.710	0.798	1 032
EWMA + block OLS	–	0.246	0.591	0.842	1 432	–	0.503	0.671	0.803	11 288

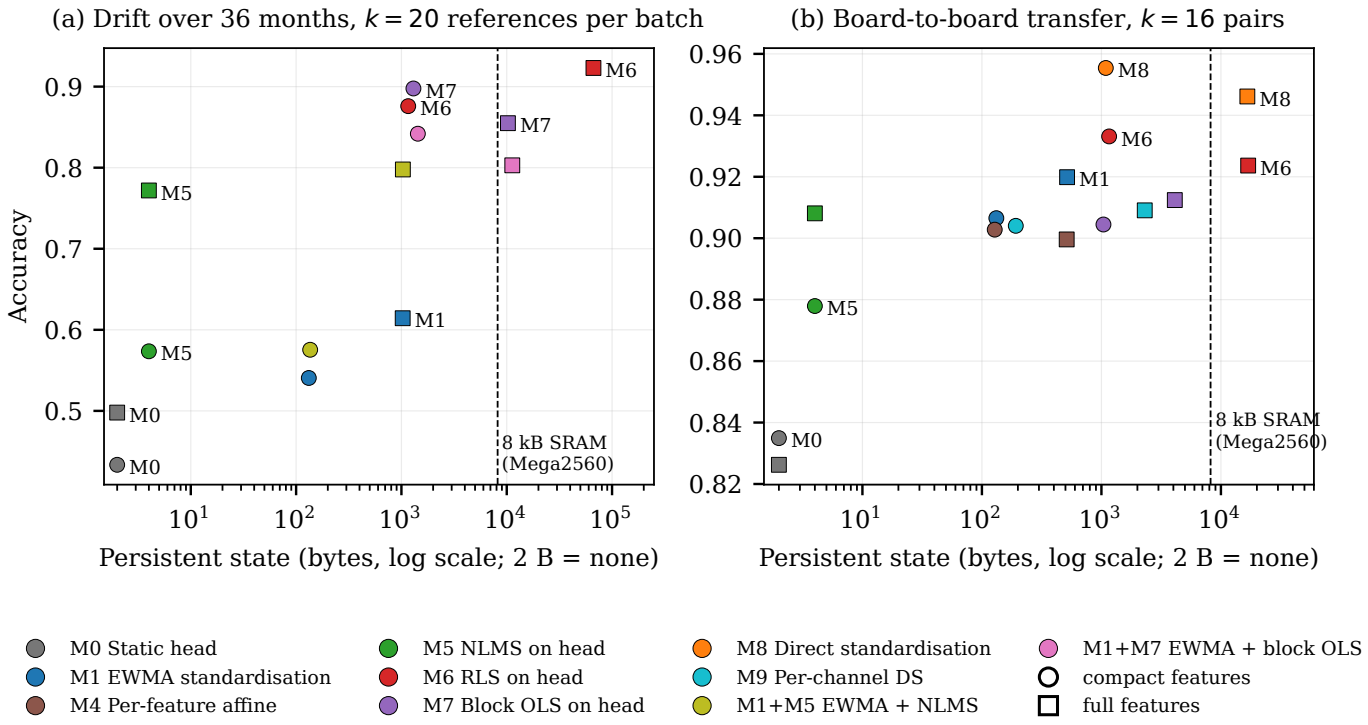


Fig. 1. Accuracy versus persistent state on the host. (a) D1 drift stream, batches 3–10, $k = 20$ references per batch. (b) D2 transfer, mean over 19 board pairs, $k = 16$ transfer pairs or references. Circles: compact features ($d = 16$); squares: full features ($d = 128$ in D1, $d = 64$ in D2). Methods without state are drawn at 2 B. The dashed line marks the 8 kB SRAM of the Mega2560, which also has to hold the head, the stack and the application.

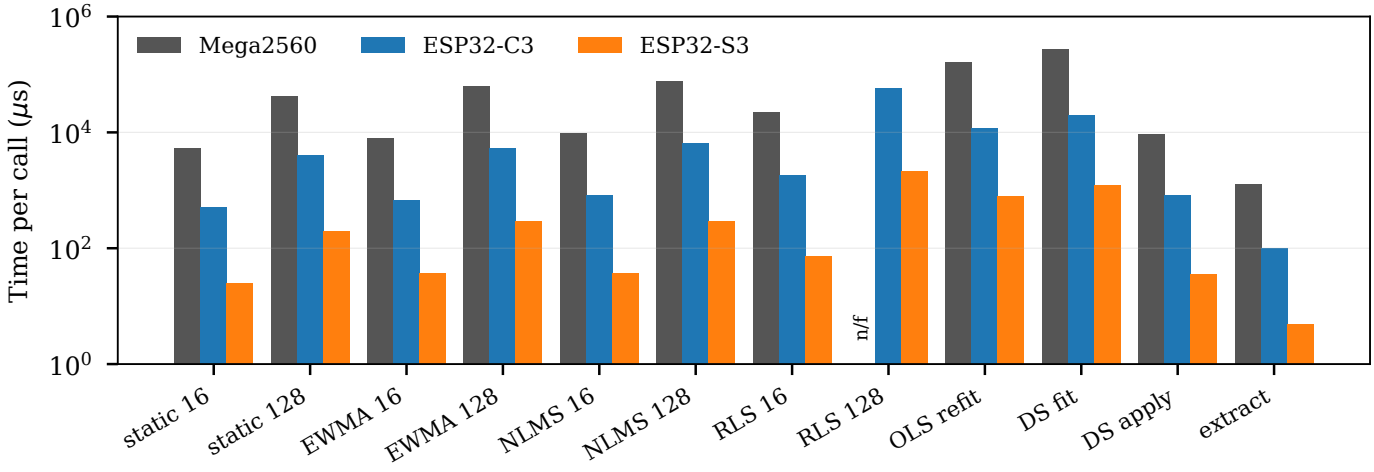


Fig. 2. Measured time per call on the three boards (median, log scale). "n/f": RLS at $d = 128$ does not fit the Mega2560. Classification and update kernels include feature preprocessing; block OLS and DS fit are executed once per calibration session.

TABLE III
CONCENTRATION REGRESSION UNDER DRIFT: RMSE IN $\log_{10}$ PPMV (BATCHES 3–10, SIX GASES, 20 SEEDS)

	$d = 16$			$d = 128$		
Method	$k = 1$	5	20	$k = 1$	5	20
Static head	0.312	0.312	0.329	0.326	0.326	0.340
EWMA standardisation	0.352	0.353	0.364	0.241	0.241	0.249
NLMS on head	0.308	0.294	0.274	0.183	0.154	0.140
RLS on head	0.283	0.193	0.322	0.182	0.148	0.317
Block OLS on head	0.224	0.184	0.174	0.176	0.138	0.119

TABLE IV
CALIBRATION TRANSFER BETWEEN TWIN BOARDS: MEAN (WORST) ACCURACY OVER 19 BOARD PAIRS, $d = 16$

Method	$k = 0$	4	8	16	State (B)
M0 static	0.831 (0.65)	0.831 (0.66)	0.832 (0.67)	0.835 (0.70)	0
M1 EWMA	0.895 (0.71)	0.899 (0.70)	0.900 (0.71)	0.907 (0.71)	132
M4 affine	–	0.889 (0.74)	0.896 (0.73)	0.903 (0.73)	128
M9 chan. DS	–	0.891 (0.74)	0.898 (0.74)	0.904 (0.74)	192
M8 DS	–	0.878 (0.75)	0.925 (0.87)	0.955 (0.92)	1 088
M5 NLMS	–	0.845 (0.72)	0.855 (0.72)	0.878 (0.72)	4
M6 RLS	–	0.878 (0.72)	0.902 (0.74)	0.933 (0.85)	1 160
M7 block OLS	–	0.848 (0.70)	0.876 (0.74)	0.904 (0.79)	1 040
M8 DS, $d = 64$	–	0.890 (0.75)	0.919 (0.77)	0.946 (0.87)	16 640

and helps both against drift and between boards. When a calibration session with about 20 known samples is possible, RLS or block least squares on 16 features give 0.88–0.90 on the 36-month stream with about 1.2 kB. When the instrument is a copy of a calibrated master and 16 paired samples are available, DS gives 0.955 with 1.1 kB. Only full-feature RLS, the most accurate method on D1, requires an ESP32-class device, and even there it costs 2.1 ms per sample on the S3 and 58 ms on the C3.

Time, in contrast, does not restrict the choice on any of the boards for the cycle times of MOX arrays. This conclusion is specific to slow chemical sensing. For faster sensors or for many arrays served by one processor, the measured times can be used directly to estimate the load.

Several limitations should be noted. The persistent state counts the adaptation state only. For methods that modify the head, the head itself (408 B at $d = 16$, 3 096 B at $d = 128$) must also reside in RAM, while static and feature-level methods can leave it in flash; peak stack and scratch memory were not measured. The protocol also foresaw Q15 replicas of the kernels and a measurement of the peak stack, which are not included in this version. Several selected hyperparameters lie on the edge of their grids (e.g., the RLS forgetting factor 0.9 in E2 and 0.8 in E3, and the NLMS step in E3); under the frozen protocol we did not re-tune them, so a sensitivity analysis would be post hoc. The references are drawn uniformly from each batch with true labels, which is optimistic compared with a calibration session that uses one gas at one concentration. The accuracy figures come from the host in float64; the boards reproduce the float32 predictions on the fixture samples only, and we did not evaluate complete streams on the boards. Finally, each board type was represented by one unit, and energy consumption and worst-case execution time were not measured.

VIII. CONCLUSION

We have measured the accuracy of linear drift-compensation and calibration-transfer methods for MOX electronic noses together with their persistent memory and their execution time on three microcontrollers. The main findings are as follows.

1) Without labels, running standardisation of the features improves the static classifier by about 11 percentage points on the 36-month drift data (0.498 to 0.614 with 1 028 B) and by 6 points between twin boards (0.831 to 0.895 with 132 B).
2) With 20 references per batch, RLS on the classifier reaches 0.923, but its state of 66.6 kB excludes an 8-bit microcontroller; on compact features, RLS and block least squares reach 0.876 and 0.898 with 1.2–1.3 kB.
3) Between twin boards, direct standardisation on 16 compact features reaches 0.955 with 1 088 B and 16 transfer pairs, as accurate as the 16.6 kB full-feature variant.
4) All kernels reproduced the host predictions on three architectures, and even on the Mega2560 the slowest per-sample kernel takes 75 ms; for gas sensor arrays, memory rather than time limits the choice of method.

The natural next step is to run complete compensation streams on the boards with a physical sensor array, and to add fixed-point kernels and energy measurements.

DATA AND CODE AVAILABILITY

Protocol, source code, results, firmware sources and the raw board logs are available in the public repository [20]. The E1/E2 protocol was frozen at commit `4a439fb`, E3/E4 at `c9799aa`; the firmware is at `578719c`, and the board captures and their analysis at `4e8adc1`. The datasets are available from the UCI repository [14], [15], [17] and are not redistributed.

ACKNOWLEDGMENT

Anthropic Claude was used to write the experiment, analysis, firmware and plotting code and to draft the manuscript under the author's direction. The author checked the generated text and code, verified the reported results against the archived records, and takes full responsibility for the content.

REFERENCES

[1] K. Persaud and G. Dodd, "Analysis of discrimination mechanisms in the mammalian olfactory system using a model nose," *Nature*, vol. 299, no. 5881, pp. 352–355, 1982.
[2] S. Marco and A. Gutiérrez-Gálvez, "Signal and data processing for machine olfaction and chemical sensing: A review," *IEEE Sensors J.*, vol. 12, no. 11, pp. 3189–3214, Nov. 2012.
[3] T. Artursson *et al.*, "Drift correction for gas sensors using multivariate methods," *J. Chemometrics*, vol. 14, nos. 5–6, pp. 711–723, 2000.
[4] M. Padilla *et al.*, "Drift compensation of gas sensor array data by orthogonal signal correction," *Chemometr. Intell. Lab. Syst.*, vol. 100, no. 1, pp. 28–35, 2010.
[5] A. Ziyatdinov *et al.*, "Drift compensation of gas sensor array data by common principal component analysis," *Sens. Actuators B, Chem.*, vol. 146, no. 2, pp. 460–465, 2010.
[6] A. Vergara, S. Vembu, T. Ayhan, M. A. Ryan, M. L. Homer, and R. Huerta, "Chemical gas sensor drift compensation using classifier ensembles," *Sens. Actuators B, Chem.*, vols. 166–167, pp. 320–329, 2012, doi: 10.1016/j.snb.2012.01.074.

TABLE V
MEASURED TIME PER CALL ON THE BOARDS (MEDIAN OF 20 TIMINGS IN TWO SESSIONS, μS) AND PERSISTENT STATE

Kernel	One call	State (B)	Host accuracy	Mega2560	ESP32-C3	ESP32-S3
Static head, $d = 16$	classify	0	0.433	5 401	505.4	25.2
Static head, $d = 128$	classify	0	0.498	42 542	4 039	197.0
EWMA, $d = 16$	classify	132	0.541	7 791	662.6	36.7
EWMA, $d = 128$	classify	1 028	0.614	61 438	5 298	289.0
NLMS, $d = 16$	classify + update	4	0.574	9 603	829.7	37.3
NLMS, $d = 128$	classify + update	4	0.772	74 769	6 528	287.2
RLS, $d = 16$	classify + update	1 160	0.876	22 102	1 805	72.8
RLS, $d = 128$	classify + update	66 568	0.923	does not fit	57 904	2 114
Block OLS, $d = 16$	refit, $k = 20$	1 300	0.898	164 426	11 650	797.0
DS, $d = 16$	fit, 16 pairs	1 088	0.955	274 708	19 792	1 224
DS, $d = 16$	map + classify	1 088	0.955	9 412	826.0	35.2
Feature extractor	one sample	426/428	–	1 265	98.1	4.79

[7] L. Zhang and D. Zhang, "Domain adaptation extreme learning machines for drift compensation in E-nose systems," *IEEE Trans. Instrum. Meas.*, vol. 64, no. 7, pp. 1790–1801, Jul. 2015.

[8] I. Rodriguez-Lujan *et al.*, "On the calibration of sensor arrays for pattern recognition using the minimal number of experiments," *Chemometr. Intell. Lab. Syst.*, vol. 130, pp. 123–134, 2014.

[9] Y. Wang, D. J. Veltkamp, and B. R. Kowalski, "Multivariate instrument standardization," *Anal. Chem.*, vol. 63, no. 23, pp. 2750–2756, 1991.

[10] J. Fonollosa *et al.*, "Calibration transfer and drift counteraction in chemical sensor arrays using direct standardization," *Sens. Actuators B, Chem.*, vol. 236, pp. 1044–1053, 2016, doi: 10.1016/j.snb.2016.05.089.

[11] [Authors to be completed], "A comprehensive review of algorithms for drift compensation in MOS gas sensor arrays," *Chemosensors*, vol. 14, no. 6, Art. no. 143, 2026.

[12] [Authors to be completed: Krayden *et al.*], "TinyML-based real-time drift compensation for gas sensors using spectral-temporal neural networks," *Chemosensors*, vol. 13, no. 7, Art. no. 223, 2025, doi: 10.3390/chemosensors13070223.

[13] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, C. Gan, and S. Han, "On-device training under 256KB memory," in *Adv. Neural Inf. Process. Syst.*, 2022. [Online]. Available: https://arxiv.org/abs/2206.15472

[14] A. Vergara, "Gas sensor array drift dataset," UCI Machine Learning Repository, 2012. [Online]. Available: https://archive.ics.uci.edu/dataset/224

[15] A. Vergara, "Gas sensor array drift dataset at different concentrations," UCI Machine Learning Repository, 2013. [Online]. Available: https://archive.ics.uci.edu/dataset/270

[16] M. K. Muezzinoglu *et al.*, "Acceleration of chemo-sensory information processing using transient features," *Sens. Actuators B, Chem.*, vol. 137, no. 2, pp. 507–512, 2009.

[17] J. Fonollosa, "Twin gas sensor arrays," UCI Machine Learning Repository, 2016, doi: 10.24432/C5MW3K.

[18] S. Haykin, *Adaptive Filter Theory*, 5th ed. Upper Saddle River, NJ, USA: Pearson, 2014.

[19] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.

[20] O. Kuznetsov, "enose-drift-mcu: Drift compensation and calibration transfer for electronic noses on microcontrollers," GitHub, 2026. [Online]. Available: https://github.com/KuznetsovKarazin/enose-drift-mcu